TP 3

Travail à faire L'objectif de ce TP est de Catégoriser un Corpus par l'implémentation d'un algorithme non supervisé (K-means)

Rappel

K-means (ou K-moyennes) :

C'est l'un des algorithmes de clustering les plus répandus. Il permet d'analyser un jeu de données caractérisées par un ensemble de descripteurs, *afin de regrouper les données "similaires" en groupes (ou* clusters*).*

La similarité entre deux données peut être inférée grâce à la "distance" séparant leurs descripteurs ; ainsi deux données très similaires sont deux données dont les descripteurs sont très proches. Cette définition permet de formuler le problème de partitionnement des données comme la recherche de ***K*** "données prototypes", autour desquelles peuvent être regroupées les autres données.

Ces données prototypes sont appelés *centroïdes ;* en pratique l'algorithme associe chaque donnée à son centroïde le plus proche, afin de créer des *clusters*. D'autre part, les moyennes des descripteurs des données d'un *cluster*, définissent la position de leur centroïde dans l'espace des descripteurs : ceci est à l'origine du nom de cet algorithme (K-moyennes ou *K-means* en anglais).

Après avoir initialisé ses centroïdes en prenant des données au hasard dans le jeu de données, *K-means* alterne plusieurs fois ces deux étapes pour optimiser les centroïdes et leurs groupes :

1. Regrouper chaque objet autour du centroïde le plus proche.
2. Replacer chaque centroïde selon la moyenne des descripteurs de son groupe.

Après quelques itérations, l'algorithme trouve un découpage stable du jeu de données : on dit que l'algorithme a convergé.

Comme tout algorithme, *K-means* présente des avantages et des inconvénients : il est simple, rapide et facile à comprendre ; cependant il ne permet pas de trouver des groupes ayant des formes complexes.

L'algorithme *k-means*

L'algorithme *k-means* se déroule en 4 étapes :

1. Choisir k objets formant ainsi k clusters
2. (Ré)affecter chaque objet O au cluster C_i de centre M_i tel que dist(O,M_i) est minimal
3. Recalculer M_i de chaque cluster (le barycentre)
4. Aller à l'étape 2 si on vient de faire une affectation

1. Importer les librairies

```
In [23]:
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score
```

2. Saisie le texte suivant

```
documents = ["This little kitty came to play when I was eating at a restaurant.",
             "Merley has the best squooshy kitten belly.",
             "Google Translate app is incredible.",
             "If you open 100 tab in google you get a smiley face.",
             "Best cat photo I've ever taken.",
             "Climbing ninja cat.",
             "Impressed with google map feedback.",
             "Key promoter extension for Google Chrome."]
```

3. Le nettoyage de données

```
In [10]:
vectorizer = TfidfVectorizer(stop_words='english')
```

4. Calculer la matrice de TF-IDF associée à notre corpus :

```
In [ ]:
X = vectorizer.fit_transform(documents)
```

5. Afficher le vocabulaire :

```
In [18]:
print(vectorizer.vocabulary_)

{'little': 18, 'kitty': 17, 'came': 4, 'play': 24, 'eating': 8, 'restau
rant': 26, 'merley': 20, 'best': 3, 'squooshy': 28, 'kitten': 16, 'bell
y': 2, 'google': 12, 'translate': 31, 'app': 1, 'incredible': 14, 'ope
n': 22, '100': 0, 'tab': 29, 'smiley': 27, 'face': 10, 'cat': 5, 'phot
o': 23, 've': 32, 'taken': 30, 'climbing': 7, 'ninja': 21, 'impressed':
13, 'map': 19, 'feedback': 11, 'key': 15, 'promoter': 25, 'extension':
9, 'chrome': 6}
```

6. Afficher les attributs :

```
In [19]: print(vectorizer.get_feature_names())

['100', 'app', 'belly', 'best', 'came', 'cat', 'chrome', 'climbing', 'e
ating', 'extension', 'face', 'feedback', 'google', 'impressed', 'incred
ible', 'key', 'kitten', 'kitty', 'little', 'map', 'merley', 'ninja', 'o
pen', 'photo', 'play', 'promoter', 'restaurant', 'smiley', 'squooshy',
'tab', 'taken', 'translate', 've']
```

7. Initialiser le nombre de cluster (la valeur de K=2)

```
In [4]: true_k = 2
        model = KMeans(n_clusters=true_k, init='k-means++', max_iter=100, n_init=1)
        model.fit(X)

Out[4]: KMeans(algorithm='auto', copy_x=True, init='k-means++', max_iter=100,
           n_clusters=2, n_init=1, n_jobs=1, precompute_distances='auto',
           random_state=None, tol=0.0001, verbose=0)
```

8. Le cluster de chaque document

```
In [26]: print(model.labels_)

[0 1 1 1 1 0 1 1]
```

9. La prédiction de la classe de document « chrome browser to open »

```
In [ ]: print("Prediction")
        Y = vectorizer.transform(["chrome browser to open."])
        prediction = model.predict(Y)
        print(prediction)
```

10. La prédiction de la classe de document « My cat is hungry »

```
In [ ]: Y = vectorizer.transform(["My cat is hungry."])
        prediction = model.predict(Y)
        print(prediction)
```

11. Afficher les attributs qui caractérisent chaque cluster (Catégorie)

In [5]:

```
print("Top terms per cluster:")
order_centroids = model.cluster_centers_.argsort()[:, ::-1]
terms = vectorizer.get_feature_names()
for i in range(true_k):
    print("Cluster %d:" % i),
    for ind in order_centroids[i, :10]:
        print(' %s' % terms[ind]),
    print
```

```
Top terms per cluster:
Cluster 0:
 climbing
 ninja
 cat
 kitty
 came
 restaurant
 play
 little
 eating
 ve
Cluster 1:
 google
 best
 translate
 app
 feedback
 impressed
 map
 incredible
 chrome
 promoter
```

12. Refaire le même travail pour k=3, k=4 …. , k=10

13. Comment peut-on optimiser la valeur de k ?